AGENTIC AI ENTERPRISE BOOTCAMP

Capstone Project Handout

Enterprise Operations Copilot — a production-style multi-agent system for internal support, compliance, and operations workflows

Student-facing brief | 4-week capstone | Premium implementation guide

What this handout is for

This brief defines the exact capstone scope, architecture, datasets, tooling, workflows, grading expectations, and packaging standards for your final submission.

Your goal is not to build a demo chatbot. Your goal is to build a system that feels credible inside a real enterprise environment.

Capstone Theme	Enterprise Operations Copilot
Primary Stack	LangGraph, FastAPI, PostgreSQL + PGVector, OpenAI API, Docker, Langfuse or LangSmith
Required Delivery	Runnable multi-container application with streaming, approval gate, async workflow, observability, and evaluation
Submission Window	4 weeks from capstone briefing date

Core principle

One enterprise platform. Multiple production concerns. One coherent system.

1. Business Scenario

You are building an internal enterprise copilot used by operations, support, and compliance teams. The system helps employees investigate cases, retrieve trusted information, coordinate across specialized agents, draft actions, and route sensitive decisions through human approval.

Example requests

Investigate why premium customer refund approvals are delayed.

Summarize the reimbursement policy and flag possible violations.

Check yesterday's P1 cases and prepare an escalation note.

Draft a customer response, but send it for manager approval first.

2. Project Objective

Build a production-style multi-agent system that can do the following:

- understand enterprise requests and route them correctly
- retrieve grounded evidence from internal documents
- use tools and APIs safely
- persist session and case state
- pause for approval on high-risk actions
- run at least one long-running task asynchronously
- stream progress updates back to the user
- capture observability traces, costs, failures, and evaluation signals
- start cleanly through Docker containers

3. Mandatory Technology Stack

Layer	Required Choice	Notes
Orchestration	LangGraph	Use for stateful graph execution, routing, pause/resume, and durable workflows.
Application API	FastAPI	Expose request, streaming, approval, and health endpoints.
LLM Provider	OpenAI API	Use for reasoning, generation, and embeddings.
Vector Store	PGVector in PostgreSQL	Store embeddings directly in Postgres and run similarity search from the same system of record.
Observability	Langfuse or LangSmith	Capture traces, latency, tool calls, failures, and token usage.
Packaging	Docker + docker compose	Your reviewer must be able to start the full stack with one command.
Optional Helpers	Hugging Face datasets, Redis, RQ/Celery	Use when they strengthen the implementation without bloating the project.

4. Required Multi-Agent Architecture

Your system must behave like a real multi-agent platform. At minimum, include the following logical responsibilities:

Supervisor Agent	Classifies the request, selects the path, and decides whether retrieval, tools, approval, or background execution are needed.
Retrieval Agent	Searches internal knowledge, applies metadata filters, and returns evidence for grounded responses.
Memory Agent	Maintains conversation summary, case memory, extracted entities, and persistent workflow state.
Tool Execution Agent	Calls APIs or internal tools with schema validation, retries, and structured outputs.
Compliance Agent	Detects unsafe content, prompt injection, PII, or policy violations before action proceeds.
Approval Agent	Pauses high-risk actions, records approval state, and resumes the workflow after a decision.
Response Composer	Builds the final grounded answer, summary, recommended action, or draft output.

Architecture expectation

This project should feel like one enterprise platform with multiple specialized capabilities, not a collection of disconnected mini-projects.

5. Required Workflows

Workflow	What it must show	Minimum expectation
Knowledge-grounded investigation	Enterprise RAG	Retrieve internal documents and produce a grounded answer with supporting evidence.
Tool-assisted investigation	Tool integration	Combine retrieved knowledge with one or more tools or APIs.
Approval-gated action	Human-in-the-loop	Pause on a high-risk action and resume only after approval or rejection.
Async/background job	Long-running execution	Offload one task such as batch review or compliance scan to a background workflow.
Guardrailed path	Safety and compliance	Show one blocked, redacted, or restricted scenario.

6. Datasets You Must Use

Use a hybrid dataset strategy. Public data gives structure, while synthetic enterprise content makes the system feel realistic.

BANKING77	Use for routing tests, intent classification, support-style requests, and supervisor evaluation.
Enron Email Dataset	Use for enterprise-style communication search, summarization, incident reconstruction, and retrieval experiments.
Synthetic Enterprise Documents	Create at least 25 internal documents such as policies, SOPs, runbooks, approval matrices, and privacy guidelines.

7. Dataset Download and Preparation Guide

Prepare your dataset pipeline as a repeatable script or notebook. Your submission must clearly show how raw data becomes retrievable content inside PostgreSQL + PGVector.

7.1 BANKING77 via Hugging Face

- Install the datasets library and load BANKING77 using load_dataset("PolyAI/banking77").
- Convert the train and test splits into DataFrames or structured records.
- Normalize fields into a schema such as query_id, user_query, intent_label, priority, department, and risk_level.
- Use this dataset mainly for routing evaluation, intent testing, and supervisor-agent benchmarking.

Expected output from this stage

A clean routing dataset that your evaluator can use to test whether the supervisor selects the correct path.

7.2 Enron Email Dataset

- Download the corpus from the official Enron dataset source and extract it locally.
- Parse each email and extract subject, sender, recipients, date, folder path, and body.
- Remove duplicates, discard low-value records, and create a combined content field for retrieval.
- Add metadata such as document_type, department, sensitivity, and source_path.

Recommended chunking standard

Chunk each document to roughly 400–800 tokens or 1,000–2,000 characters with overlap. Store metadata with every chunk so retrieval can be filtered and explained.

7.3 Synthetic Enterprise Corpus

- Create at least 25 internal documents in Markdown, text, or JSON format.
- Suggested files include refund_policy, reimbursement_policy, escalation_sop, approval_matrix, pii_guidelines, incident_runbook, and org_directory.
- Treat these documents as the internal knowledge base that makes your capstone feel like a believable company system.

8. Embedding and PGVector Requirements

- Use OpenAI embeddings for vector creation.
- Recommended default model: text-embedding-3-small.
- Store chunk embeddings inside PostgreSQL using the PGVector extension.
- Maintain a document table and a document_chunks table with metadata fields.
- Implement similarity search and show how top results are fed into the response path.

Minimum schema expectation

documents table for source-level records
document_chunks table for chunk_text, metadata, and embedding vector
sessions, cases, and approvals tables for workflow persistence

9. Tools, Memory, and Workflow State

Implement at least five tools. A strong default set is ticket lookup, policy lookup, org lookup, approval submission, and notification or draft creation.

- Persist session memory, conversation summary, case state, extracted entities, and approval status.
- Design state in a way that the graph can pause and resume safely.
- All tool outputs should be structured and recoverable. Handle retries and failure reporting explicitly.

10. Async and Human-in-the-Loop Requirements

Capability	Required behavior	Examples
Async job	At least one task must execute in the background.	Batch ticket triage, long document review, compliance scan.
Approval gate	At least one risky action must pause for approval.	Refund recommendation, escalation draft, policy exception request.
Streaming	Progress or answer stream must be visible to the user.	Classifying request, retrieving documents, waiting for approval.

11. Guardrails and Observability

- Implement input validation, PII detection or masking, output validation, and restricted-action handling.
- Demonstrate one blocked or redacted scenario in the final demo.
- Use Langfuse or LangSmith to capture traces for model calls, retrieval steps, tool calls, latency, failures, token usage, and estimated cost.

Non-negotiable principle

If a reviewer cannot see what the system did, why it did it, and where it failed, the system is not production-ready.

12. Testing and Evaluation

- Write unit tests for parsers, tools, validation logic, and helper utilities.
- Write integration tests for end-to-end flows including retrieval, tool usage, and approval pause/resume.
- Prepare at least 20 evaluation scenarios covering routing correctness, retrieval usefulness, groundedness, action safety, and approval behavior.

13. Dockerization and Runtime Standard

- Package the application with a Dockerfile and a docker compose configuration.
- At minimum, include the app service and PostgreSQL with PGVector enabled.
- Optional services such as Redis or observability containers may be included if they strengthen the setup.
- Your README must clearly explain how to start the full stack and verify readiness.

Success criterion

A reviewer should be able to run docker compose up --build and bring up the system without manually wiring the environment from scratch.

14. Required Deliverables

Deliverable	What to include	Why it matters
GitHub repository	Code, README, Docker files, env example, tests, and run instructions.	Makes the project reproducible.
Architecture diagram	Graph, agents, APIs, database, vector store, and observability.	Shows system design maturity.
Dataset prep script	Loading, cleaning, chunking, embedding, and storage pipeline.	Proves you can operationalize data, not just talk about it.
Working application	All required workflows implemented.	Demonstrates end-to-end execution.
Technical write-up	Use case, trade-offs, risks, and improvement plan.	Makes the system interview-defensible.
Demo video	10–15 minute walkthrough of core flows.	Lets a reviewer validate the product story quickly.
Evaluation report	Scenarios, outcomes, failure analysis,	Shows production thinking.

	and next steps.	

15. Milestones

Week 1	Define the use case, prepare datasets, sketch the architecture, and finalize schema and tools.
Week 2	Build the graph, retrieval pipeline, memory layer, and first end-to-end workflow.
Week 3	Add approval gate, async execution, guardrails, tests, and observability.
Week 4	Dockerize, finalize evaluation, record the demo, and package the full submission.

16. Grading Rubric

Category	Weight	Reviewer focus
Enterprise realism	10	Does the problem and solution feel believable in a real company?
Multi-agent architecture	15	Are responsibilities clear, justified, and well orchestrated?
RAG + PGVector	15	Is retrieval useful, grounded, and well implemented?
Tools and APIs	10	Are integrations structured, safe, and failure-aware?
Async + HITL	10	Does the system truly support background execution and approval gating?
Guardrails + safety	10	Are risky or unsafe behaviors constrained properly?
Deployment + streaming	10	Is the system runnable, responsive, and containerized cleanly?
Observability + evals	15	Can reviewers inspect traces, failures, and quality metrics?
Documentation + demo	5	Is the final story clear and defensible?

17. What Strong Submissions Have in Common

- One clear enterprise problem, not five shallow ideas.
- Agent boundaries that feel necessary, not decorative.
- Grounded retrieval, visible traces, and explainable decisions.
- A real approval path and a real long-running workflow.
- A Dockerized system that starts cleanly and tells a coherent product story.

18. Final Submission Checklist

- I used LangGraph, OpenAI API, PostgreSQL + PGVector, and FastAPI.

- I implemented at least five tools.
- I loaded public data and created synthetic enterprise documents.
- I implemented memory, an async workflow, an approval workflow, and streaming.
- I added guardrails and observability.
- I dockerized the project and documented startup clearly.
- I created a technical write-up, evaluation report, and demo video.

Capstone reminder

Do not optimize for novelty. Optimize for coherence, reliability, and production-style execution. Systems ship. Demos do not.